

Aptimizer

A plain-English guide to everything the software is built from — written for a reader who has never used any of these tools.

What the software actually is

Aptimizer is a web application for real-estate and civil-engineering project planning in India. An architect or developer draws a plot of land, enters what they want to build on it, and the software works out whether it is legal, whether it stands up, what it costs, how long it takes and whether it makes money — then writes the reports.

It is used through a normal web browser. There is nothing to install. You sign in, create a project, and everything lives inside that project.

What it does, in order

- **Compliance** — checks the scheme against Indian building rules (IS codes, the National Building Code, local development control regulations): setbacks, floor-area ratio, height limits, parking counts, fire access.
- **Engineering** — sizes the structure. Columns, beams, slabs, foundations, earthquake and wind loads, each answer citing the exact code clause it came from.
- **Layout** — generates floor plans automatically: it places rooms inside each flat against the outside wall, and places the towers on the site.
- **Quantities and cost** — counts the concrete, steel, bricks and finishes the design implies and turns that into a bill of quantities with prices.
- **Schedule** — builds a construction programme where task durations are calculated from the quantities, not typed in by hand.
- **Finance** — sale value, profit, cash flow, and when the money actually arrives.
- **Site intelligence** — reads real map data around the plot: roads, terrain height, flood and wind exposure, sun path, what is nearby.
- **APT** — a built-in AI assistant that can explain any number on screen, and write reports.

How to read the rest of this document

A web application is always made of the same few layers. Each section below explains one layer in everyday terms first, then lists the specific tools Aptimizer uses in that layer and what each one is for. Nothing assumes prior knowledge.

The big picture: five layers

Imagine a restaurant. The **dining room** is what customers see and touch. The **kitchen** does the actual work. The **store room** holds the ingredients between services. The kitchen sometimes phones **outside suppliers** for things it cannot make itself. And the **building** houses all of it.

Layer	Restaurant	In software	In Aptimizer
Frontend	The dining room	The screens in your browser: buttons, forms, charts, maps	React, Tailwind, Leaflet, Recharts, three.js
Backend	The kitchen	The programs on a server that do the calculations and enforce the rules	Python with FastAPI, plus about 20 calculation modules
Database	The store room	Where information is kept so it is still there tomorrow	MongoDB
APIs	Phone lines — internal and to suppliers	Fixed request formats: one program asking another for something	About 70 internal endpoints; several outside services
Infrastructure	The building and its plumbing	How all of it is packaged, run and kept online	Docker, Docker Compose, Caddy, Render

The one sentence that ties it together

You click something in the **frontend**; it sends a request over an **API** to the **backend**; the backend calculates, reads or writes the **database**, sometimes calls an **outside service**, and sends an answer back; the frontend draws that answer on screen. Every single feature in Aptimizer is that same loop.

1. The frontend — what you see

The frontend is the part that runs inside your browser. It is written in JavaScript and its whole job is to show things and collect input. It performs no engineering calculations of its own; it asks the backend for every number it displays.

Optimizer has seven screens: a landing page, sign-in and registration, a projects list, the main **Workspace** where all the work happens, a profile page, an admin page, and a public compliance page that can be shared with someone who has no account.

The tools used, and why

Tool	What it is	Why it is here
React	The most widely used library for building web interfaces. It lets a page be assembled from reusable pieces called components.	The Workspace is one screen with many panels. React lets each panel be built once and reused, and redraws only what changed.
React Router	Handles which screen you are on.	Gives every page its own web address, so the browser Back button and shareable links work.
Tailwind CSS	A styling system: appearance is set with small named classes instead of separate style files.	Keeps spacing, colour and type consistent across a large app without a designer redrawing each screen.
shadcn/ui + Radix	A set of ready-made interface parts — dialogs, dropdowns, tabs, sliders, tooltips — built to work with a keyboard and a screen reader.	Nobody rebuilds a dropdown from scratch. These are correct and accessible from the start.
Leaflet + react-leaflet	A mapping library that shows an interactive map you can pan and zoom.	This is where you draw the plot boundary and see the site in context.
Recharts	A charting library.	Draws the cost breakdowns, cash-flow curves and schedule charts.
three.js + React Three Fiber	A 3D graphics engine for the browser.	Shows the towers and massing in three dimensions rather than only on plan.
React Hook Form + Zod	Form handling and input validation.	Catches a bad entry — a negative plot area, a missing floor count — before it is ever sent to the server.
TanStack Query, SWR, Axios	Three tools for fetching data from the backend and remembering it.	They handle loading states, retries and caching, so the same project data is not requested again and again.
Framer Motion / Motion	Animation.	Panels slide and fade instead of jumping, which makes state changes readable.
Lucide, Sonner, cmdk, Vault	Icons, pop-up notifications, a command palette, mobile drawers.	Small, standard interface conveniences.
CRA + CRACO	The build setup. It turns the source code into files a browser can load.	CRACO is a thin add-on that lets the standard setup be customised without replacing it.

2. The backend — where the work happens

The backend is a program running on a server. It receives requests from the frontend, does the actual engineering, checks permissions, saves and loads data, and sends answers back. It is written in **Python**, the language most used for scientific and numerical work.

The framework

Tool	What it is	Why it is here
FastAPI	A Python framework for building the request-and-answer machinery of a web service.	It is fast, it validates every incoming request automatically, and it documents itself.
Uvicorn	The server process that actually listens on the network and hands requests to FastAPI.	FastAPI is the recipe; Uvicorn is the stove.
Pydantic	Describes the shape of the data — what a Project contains, what a Tower contains.	If a request arrives with a missing or wrong-typed field, it is rejected with a clear message rather than crashing halfway through a calculation.

The calculation modules

Beyond the plumbing, the backend is roughly 12,000 lines of domain code split into about twenty focused files. These are the actual product — everything else in this document exists to carry their answers to the screen.

Module	What it does
iscodes	The single source of truth for every Indian Standard and NBC constant and clause used anywhere in the app. No other file hardcodes a code value, so a code revision is a one-file change.
engine	The core analysis: takes the project and produces areas, FAR, coverage, unit mix, compliance verdicts.
engineering	Fourteen IS-code and NBC driven structural modules — columns, beams, slabs, foundations, seismic and wind — each answer carrying its clause reference and reporting missing inputs rather than guessing.
layout	Generates room layouts inside a flat, placing every room that needs a window against the exterior wall, with balconies in front.
parking	Parking demand and supply per building, scaled by dwelling size the way Indian bye-laws actually do, rather than one flat site-wide ratio.
takeoff	Quantity take-off — the concrete, steel and masonry the designed structure implies, derived from the real column grid and slab thicknesses, not thumb rules.
optimise / planopt	The optimisers. Each answers three things: what the scheme does now, what the best version found does, and the specific changes between them. planopt scores every candidate by re-running the real analysis on it.
schedule	Turns quantities into activities, then into a critical-path programme with dates, float and cash flow. Durations are derived from quantity and crew output, so changing 12 floors to 18 re-plans the job automatically.

Module	What it does
finance	Development finance: sale value, cost of borrowing, margin, and the timing of cash in and out.
gis	Site intelligence from real map data: nearby features, terrain height, flood, wind, sun path, accessibility, and a buildability score.
reports	Builds the PDF and Excel deliverables.
citations	Checks every code clause the AI cites against the real clause registry, because a confident wrong citation is worse than none.
ai / aptcontext / aptspeed / aptsuggest	The APT assistant: the single place the app talks to an AI model, what project context is sent, how that is kept small and fast, and the opening questions offered for each screen.
auth / turnstile	Sign-in, passwords, sessions, and bot protection on the two public pages.

Supporting Python libraries

Library	What it is for
NumPy, Pandas	Fast numerical arrays and tables — the mathematics underneath the engine.
Shapely	Geometry: plot polygons, setback offsets, overlaps, areas.
ReportLab	Generates the PDF reports.
OpenPyXL	Generates the Excel bill-of-quantities file.
Motor / PyMongo	The drivers that let Python talk to the database.
PyJWT, bcrypt, passlib	Sign-in tokens and password protection (see below).
httpx, aiohttp, requests	Calling outside services over the internet.
LiteLLM	One uniform way to talk to several different AI providers.
pytest, black, flake8, mypy	Testing and code-quality tools used during development.

3. The database — where things are kept

A program forgets everything the moment it stops. A database is separate storage that survives restarts. When you close the browser and come back next week, your project is still there because it was written to the database.

MongoDB

Optimizer uses **MongoDB**. Most people picture a database as a spreadsheet — fixed columns, every row the same shape. MongoDB is the other kind: it stores **documents**, which are more like nested forms, and two documents in the same collection need not have identical fields.

That fits this app well. One project may have three towers with generated floor layouts, a full schedule and a GIS run; another may be a plot sketch with nothing calculated yet. Both are just projects. A rigid table would need dozens of mostly-empty columns.

What is stored

Collection	What lives in it
projects	The heart of it: the plot, the towers, every input and every computed result.
users	Accounts — email, name, role, and the scrambled form of the password.
versions	Saved snapshots of a project, so you can compare schemes or roll back.
shares	Who a project has been shared with, and any public read-only link.
activity	An audit trail of what changed and who changed it.
login_attempts	Recent sign-in attempts, used to slow down password guessing.

Motor is the driver used to reach it. It is asynchronous, meaning the server can start a database read and get on with other requests while waiting, instead of standing idle.

4. APIs — how the parts talk

An **API** (Application Programming Interface) is simply an agreed way for one program to ask another for something. Like a restaurant menu: fixed items, fixed names, a known response. You cannot ask for anything not on the menu, and that is the point — both sides know exactly what to expect.

An **endpoint** is one item on that menu: one address plus one verb. *GET* means fetch something, *POST* means create or run something, *PUT* means update, *DELETE* means remove.

Optimizer's own API

Every address starts with **/api**. There are about seventy of them. Grouped by purpose:

Group	Examples	What it does
Accounts	/auth/register, /auth/login, /auth/refresh, /auth/me	Create an account, sign in, keep the session alive, ask who you are.
Projects	GET and POST /projects, PUT and DELETE /projects/{id}	List, open, create, rename and delete projects.
Analysis	POST /analyse, /engineering/analyse, GET /projects/{id}/analysis	Run the compliance and structural engines and read back the results.
Layout	/site-layout/envelope, /site-layout/plan, /site-layout/recommend, /towers/{id}/floor-layout	Work out the buildable envelope, place towers, generate floor plans.
Cost & time	POST /projects/{id}/optimise, /schedule, /finance	Optimise quantities and cost, build the programme, run the financial appraisal.
Site data	POST /projects/{id}/gis/analyse, GET /projects/{id}/gis	Fetch and analyse the real-world data around the plot.
Deliverables	GET /projects/{id}/boq.xlsx, /projects/{id}/reports/{type}	Download the bill of quantities as Excel and the reports as PDF.
Versions	/projects/{id}/versions, /versions/compare, /versions/{id}/restore	Snapshot, compare and roll back schemes.
Sharing	/projects/{id}/shares, /public-link, /public/compliance/{token}	Share with a colleague, or publish a read-only compliance page anyone can open.
APT assistant	/ai/chat, /ai/chat/stream, /ai/compliance, /ai/cost, /ai/report, /ai/compare	Ask the assistant questions and have it write reports. The <i>stream</i> one sends the answer word by word as it is written.
Reference data	/iscodes, /defaults, /cities, /finance/defaults, /schedule/defaults	Code constants and sensible starting values so a new project is not a blank form.

Outside APIs the backend calls

Some things the app cannot know by itself — where the roads are, how high the ground is, whether a visitor is a robot. For those it calls other people's services over the internet.

Service	What it provides	Used for
OpenStreetMap Overpass	A free, community-built map of the world that can be queried for specific features. Three separate mirrors are configured so the app keeps working if one is down.	Finding roads, buildings, water bodies and amenities around the plot.
Open-Elevation	Ground height for any coordinate.	Terrain slope, drainage, flood exposure.
OSM & ArcGIS map tiles	The map images themselves — street maps and satellite imagery.	The background of the map you draw the plot on.
Cloudflare Turnstile	A modern replacement for those 'select all traffic lights' puzzles. It scores the visitor silently and usually shows nothing at all.	Protecting sign-in and registration. Those are the only pages a stranger can post to, and an unprotected one lets a script create endless accounts and burn the AI budget.
Google Gemini	An AI model.	Powers APT and the written reports.
xAI Grok	A second AI model.	Alternative provider.
Groq	A third provider, notable for very fast responses.	Alternative provider.

Why three AI providers

All AI calls in the app go through a single file. It tries providers in a set order and steps down to a smaller model if the first is unavailable. If no key is configured at all, the AI features switch themselves off with a plain explanation instead of showing an error. The provider is a setting, not a rewrite.

5. Security — who is allowed to do what

Piece	In plain terms
Password hashing (bcrypt)	Passwords are never stored. They are put through a one-way scramble; sign-in re-scrambles what you typed and compares. Even someone holding the whole database cannot read them back.
JWT tokens	After you sign in, the server gives your browser a signed pass. Every later request carries it, so you are not re-typing a password all day. It is signed with a secret key, so a forged one is rejected.
Refresh tokens	The pass expires quickly for safety; a longer-lived refresh token quietly gets a new one so you are not thrown out mid-session.
Roles	Accounts have roles. The admin page and the user list are visible only to admins.
Login-attempt tracking	Repeated failures are recorded and slowed, which defeats automated password guessing.
Turnstile	Bot protection on the two pages a stranger can reach.
Share tokens	A public compliance link is a long random token, not a guessable project number, and it grants read-only access to that one page.

6. Infrastructure — how it runs

Tool	What it is	Why it is here
Docker	Packages a program together with everything it needs into a sealed box called a container, which runs the same on any machine.	Removes 'it works on my computer'. There is one box for the backend, one for the frontend, one for the database.
Docker Compose	Describes and starts several containers together with one command.	<i>docker compose up</i> brings up the whole application — database, backend and web app — wired together.
Caddy	A web server that obtains and renews HTTPS certificates by itself.	Optional, switched on for a real public domain so the site is served securely with no manual certificate work.
Render	A hosting service.	A configuration file is included so the app can be deployed there instead of a self-managed server.
Environment variables	Settings kept outside the code — the database address, the secret key, the AI keys.	Secrets never sit in the source code, and the same code runs locally and in production with different settings.

A note on MCPs

MCP stands for Model Context Protocol — a recent standard that lets an AI assistant plug into outside tools and data sources in a uniform way. It is worth knowing the term because it comes up often, but **Optimizer does not use any MCPs**.

Its AI features work the older, simpler way: the backend assembles the relevant project figures itself, sends them to the model as part of the question, and gets text back. The model is never given tools to run or a database to reach into. The only MCP-shaped things on the machine belong to the developer's own editor, not to the product.

If you remember only one page

Term	One-line meaning
Frontend	The screens in your browser. React.
Backend	The server programs that calculate and enforce rules. Python and FastAPI.
Database	Permanent storage. MongoDB.
API	An agreed request format one program uses to ask another for something.
Endpoint	One specific item on that menu, e.g. POST /api/schedule.
Library	Someone else's reusable code you build on instead of writing it yourself.
Framework	A larger library that also sets the overall shape of your program.
Container	A sealed box holding a program and everything it needs to run.
Token	A signed pass proving who you are, issued after sign-in.
MCP	A standard for connecting AI assistants to tools. Not used here.